Open in Colab

Arithmetic Functions

`abs(number, /)`

Return the absolute value of a number. The argument may be an integer, a floating-point number, or an object implementing **abs**(). If the argument is a complex number, its magnitude is returned.

`pow(base, exp, mod=None)`

Return base to the power `exp`; if `mod` is present, return base to the power `exp`, modulo `mod` (computed more efficiently than `pow(base, exp) % mod`). The two-argument form `pow(base, exp)` is equivalent to using the power operator: `base**exp`.

`divmod(a, b, /)`

Take two (non-complex) numbers as arguments and return a pair of numbers consisting of their quotient and remainder when using integer division. With mixed operand types, the rules for binary arithmetic operators apply. For integers, the result is the same as `(a // b, a % b)`. For floating-point numbers the result is `(q, a % b)`, where `q` is usually `math.floor(a / b)` but may be 1 less than that. In any case `q * b + a % b` is very close to `a`, if `a % b` is non-zero it has the same sign as `b`, and `0 <= abs(a % b) < abs(b)`.

`round(number, ndigits=None)`

Return `number` rounded to `ndigits` precision after the decimal point. If `ndigits` is omitted or is `None`, it returns the nearest integer to its input.

Functions on Iterables

In [21]:
```
abs(-3+4j)
```

Out[21]:
```
5.0
```

`all(iterable, /)`

Return `True` if all elements of the iterable are true (or if the iterable is empty). Equivalent to:

In [22]:
```
def all(iterable):
    for element in iterable:
        if not element:
            return False
    return True
```

`any(iterable, /)`

Return True if any element of the iterable is true. If the iterable is empty, return False. Equivalent to:

In [23]:
```
def any(iterable):
    for element in iterable:
        if element:
            return True
    return False
```

`max(iterable, /, *, default, key=None)`

`max(arg1, arg2, /, *args, key=None)`

Return the largest item in an iterable or the largest of two or more arguments.

`min(iterable, /, *, default, key=None)`

`min(arg1, arg2, /, *args, key=None)`

Return the smallest item in an iterable or the smallest of two or more arguments.

`sum(iterable, /, start=0)`

Sums `start` and the items of the `iterable` from left to right and returns the total. The `iterable`'s items are normally numbers, and the start value is not allowed to be a string.

`filter(function, iterable, /)`

Construct an iterator from those elements of iterable for which function is true. iterable may be either a sequence, a container which supports iteration, or an iterator. If function is None, the identity function is assumed, that is, all elements of iterable that are false are removed.

map(function, iterable, /, *iterables, strict=False)``

Return an iterator that applies function to every item of iterable, yielding the results. If additional iterables arguments are passed, function must take that many arguments and is applied to the items from all iterables in parallel. With multiple iterables, the iterator stops when the shortest iterable is exhausted.

`reversed(iterable, /)`

Return a reverse iterator.

`sorted(iterable, /, *, key=None, reverse=False)`

Return a new sorted list from the items in iterable.

`enumerate(iterable, start=0)`

Return an enumerate object. iterable must be a sequence, an iterator, or some other object which supports iteration. The `__next__()` method of the iterator returned by `enumerate()` returns a tuple containing a count (from start which defaults to 0) and the values obtained from iterating over iterable.

In [24]:
```
months = [
"January",
"February",
"March",
"April",
"May",
"June",
"July",
"August",
"September",
"October",
"November",
"December",]
list(enumerate(months,start=1))
```

Out[24]:
```
[(1, 'January'),
 (2, 'February'),
 (3, 'March'),
 (4, 'April'),
 (5, 'May'),
 (6, 'June'),
 (7, 'July'),
 (8, 'August'),
 (9, 'September'),
 (10, 'October'),
 (11, 'November'),
 (12, 'December')]
```

`zip(*iterables, strict=False)`

Iterate over several `iterables` in parallel, producing tuples with an item from each one. For example:

In [25]:
```
for item in zip([1, 2, 3], ['sugar', 'spice', 'everything nice']):
    print(item)
```

```
(1, 'sugar')
(2, 'spice')
(3, 'everything nice')
```

Other Builtin Functions

`len(object, /)`

Return the length (the number of items) of an object.

`eval(source, /, globals=None, locals=None)`

The source argument is parsed and evaluated as a Python expression.

In [26]:
```
eval('2**5+1')
```

Out[26]: 33

`exec(source, /, globals=None, locals=None, *, closure=None)`

This function supports dynamic execution of Python code. `source` is parsed as a suite of Python statements which is then executed.

In [27]:
```
exec("""
if True:
  print("Hello")
""")
```

```
Hello
```

`callable(object, /)`

Return True if the object argument appears callable, False if not.

`ord(character, /)`

Return the ordinal value of a character.

If the argument is a one-character string, return the Unicode code point of that character. For example, `ord('a')` returns the integer `97` and `ord('€')` (Euro sign) returns `8364`. This is the inverse of `chr()`.

`chr(codepoint, /)`

Return the string representing a character with the specified Unicode code point. For example, `chr(97)` returns the string `'a'`, while `chr(8364)` returns the string `'€'`. This is the inverse of `ord()`.

`help(request)`

Invoke the built-in help system. (This function is intended for interactive use.) If no argument is given, the interactive help system starts on the interpreter console. If the argument is a string, then the string is looked up as the name of a module, function, class, method, keyword, or documentation topic, and a help page is printed on the console. If the argument is any other kind of object, a help page on the object is generated.

`type(name, bases, dict, /, **kwargs)`

With one argument, return the type of an object.

`hash(object, /)`

Return the hash value of the object (if it has one). Hash values are integers. They are used to quickly compare dictionary keys during a dictionary lookup. Numeric values that compare equal have the same hash value (even if they are of different types, as is the case for `1` and `1.0`).

```
In [28]: hash("Hello")
```

```
Out[28]: 2927047315895827246
```

`print(*objects, sep=' ', end='\n', file=None, flush=False)`

Print objects to the text stream file, separated by sep and followed by end. sep, end, file, and flush, if present, must be given as keyword arguments.

`input(prompt, /)`

If the prompt argument is present, it is written to standard output without a trailing newline. The function then reads a line from input, converts it to a string (stripping a trailing newline), and returns that

`open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)`

Open file and return a corresponding file object.

Functions For Attributes

`dir(object, /)`

Without arguments, return the list of names in the current local scope. With an argument, attempt to return a list of valid attributes for that object.

`getattr(object, name, default, /)`

Return the value of the named attribute of object. name must be a string. If the string is the name of one of the object's attributes, the result is the value of that attribute. For example, `getattr(x, 'foobar')` is equivalent to `x.foobar`.

`setattr(object, name, value, /)`

This is the counterpart of `getattr()`. The arguments are an object, a string, and an arbitrary value. The string may name an existing attribute or a new attribute. The function assigns the value to the attribute, provided the object allows it. For example, `setattr(x, 'foobar', 123)` is equivalent to `x.foobar = 123`.

hasattr(object, name, /)``

The arguments are an object and a string. The result is `True` if the string is the name of one of the object's attributes, `False` if not.

Formatting Output

`repr(object, /)`

Return a string containing a printable representation of an object.

`str(object, *, errors)`

Return a string version of object. If object is not provided, returns the empty string.

`bin(integer, /)`

Convert an integer number to a binary string prefixed with " `0b` ". The result is a valid Python expression. If integer is not a Python int object, it has to define an `__index__()` method that returns an integer. Some examples:

`hex(integer, /)`

Convert an integer number to a lowercase hexadecimal string prefixed with "0x". If integer is not a Python int object, it has to define an `__index__()` method that returns an integer. For example:

```
In [29]: hex(~123 & 0xffff)
```

```
Out[29]: '0xff84'
```

`oct(integer, /)`

Convert an integer number to an octal string prefixed with " `0o` ". The result is a valid Python expression. For example:

```
In [30]: oct(12)
```

Out[30]: '0o14'

`format(value, format_spec='', /)`

Convert a value to a “formatted” representation, as controlled by format_spec. The interpretation of format_spec will depend on the type of the value argument; however, there is a standard formatting syntax that is used by most built-in types: Format Specification Mini-Language.

The default format_spec is an empty string which usually gives the same effect as calling str(value).